EEE 448

Reinforcement Learning & Dynamic Programming

PROJECT REPORT II

Dynamic Programming and Reinforcement Learning Solutions

Bilkent University
Çağan İzol
22303351
Electrical and Electronics Engineering Department
Course Instructor: Muhammed O. Sayin
Spring 2026

Prepared for Tasks 3 and 4 only.
Tasks 1 and 2 are not repeated; their MDP and simulator definitions are used as the foundation.

1 Introduction and Scope

This report completes Tasks 3 and 4 of the EEE 448 project. The purpose is to solve the finite tabular trading MDP formulated previously, first with dynamic programming using the known model and then with model-learning and model-free reinforcement-learning methods using the simulator.

The report intentionally does not copy the full derivations from Tasks 1 and 2. Instead, it uses their validated state, action, transition, reward, and simulation definitions as inputs. The numerical configuration remains the standardized setup: x0 = 10, B0 = 30, m0 = 0, initial market state = Sideways, price cap x_bar = 20, budget cap Bmax = 50, holdings cap mmax = 10, and discount factor gamma = 0.95.

All performance values in the main comparison are discounted returns from the initial state s0 = (x=10, B=30, m=0, mu=Sideways). Experimental performance is measured by Monte Carlo simulation using 100,000 independent trials unless a table explicitly states another trial count. A rollout horizon of T = 2000 is used; after the price reaches x=0, future rewards are zero, so this horizon is effectively infinite for the reported returns.

Item	Value / Convention	Reason
State	s = (x, B, m, mu)	Price, liquid budget, holdings, and market regime are required for Markovian control.
State-space size	21 x 51 x 11 x 3 = 35,343	Finite tabular solution is possible.
Action	Integer a: buy if a>0, hold if a=0, sell if a<0	Trades occur before the price update.
Reward	$r_k = (m_k + a_k)\,(x_{k+1} - x_k)$	Change in wealth after the post-trade position is exposed to the next price move.
Bankruptcy event	$W_k = B_k + m_k x_k = 0$	Matches the assignment definition of wealth becoming zero.
Stochastic model	Market transition plus price noise	Known for Task 3.1; estimated or sampled for Tasks 3.3 and 4.

Table 1.1 - Numerical and modelling conventions used in Report II.

2 Task 3 - Dynamic Programming Solutions

2.1 Bellman Equations for the Discounted Infinite-Horizon MDP

The dynamic-programming solution is based on the Bellman optimality equation. For every feasible state s and action a in A(s), the action-value expression is

$$Q(s,a) = \sum_{s'} P(s' \mid s,a)\,[\,r(s,a,s') + \gamma V(s')\,]$$

The optimal value function and optimal stationary policy satisfy

$$V^*(s) = \max_{a \in A(s)} Q(s,a)$$

$$\pi^*(s) \in \arg\max_{a \in A(s)} Q(s,a)$$

Because gamma = 0.95 < 1 and the state and action sets are finite, the Bellman optimality operator is a contraction. Therefore value iteration converges to the unique fixed point V*, and policy iteration converges to an optimal stationary deterministic policy.

2.2 Transition and Reward Evaluation Used by the Algorithms

For a current state s = (x, B, m, mu) and feasible action a, the budget and holdings after trading are deterministic:

$$\mathrm{B+} = \mathrm{B} - \mathrm{a\,x}, \qquad \mathrm{m+} = \mathrm{m} + \mathrm{a}.$$

The next market state μ' is drawn from $P_M(\mu, \mu')$,while the next price x' is obtained from the simulator price update using the noise distribution conditioned on the current market state mu. Hence the one-step expectation used inside every Bellman backup is

$$Q(s,a) = \sum_{\omega} \sum_{\mu'} P_{\text{noise}}(\omega \mid \mu)\, P_M(\mu,\mu')[m^+(\text{f(x,ω)} - \text{x}) + \gamma \text{V(f(x,ω)},B^+,m^+,\mu')]$$

The same convention as in the simulator is used: the noise depends on the current market state before the market transition. This detail is important because using mu' instead of mu would solve a different MDP.

2.3 Task 3.1.1 - Value Iteration

Value iteration starts withV_0(s)=0 for all states and repeatedly applies the Bellman optimality operator. At iteration n,

$$V_{n+1}(s) = \max_{a \in A(s)} \sum_{s'} P(s' \mid s,a)[\text{r(s,a,}s') + \gamma V_n(s')]$$

The stopping criterion used in the numerical implementation is

$$\max_{s} \mid V_{n+1}(\text{s}) - V_n(\text{s}) \mid < 10^{-8}$$

The action attaining the maximum in the final Bellman backup is stored as the value-iteration policy. For states with multiple exactly equal maximizing actions, the first maximizing feasible action is selected consistently. This tie-breaking has no effect on the value but fixes a deterministic policy.

Quantity	Result
Number of states	35,343
Maximum feasible actions per state	11
Discount factor	0.95
Stopping tolerance	1e-8
Value-iteration sweeps	284
Final maximum value change	9.54e-9
Optimal value at s0	$V^*(s_0) = 14.2808$
Optimal action at s0	$a^*(10,30,0,\text{Sideways}) = 0$ (hold)

Table 3.1 - Value iteration numerical results.

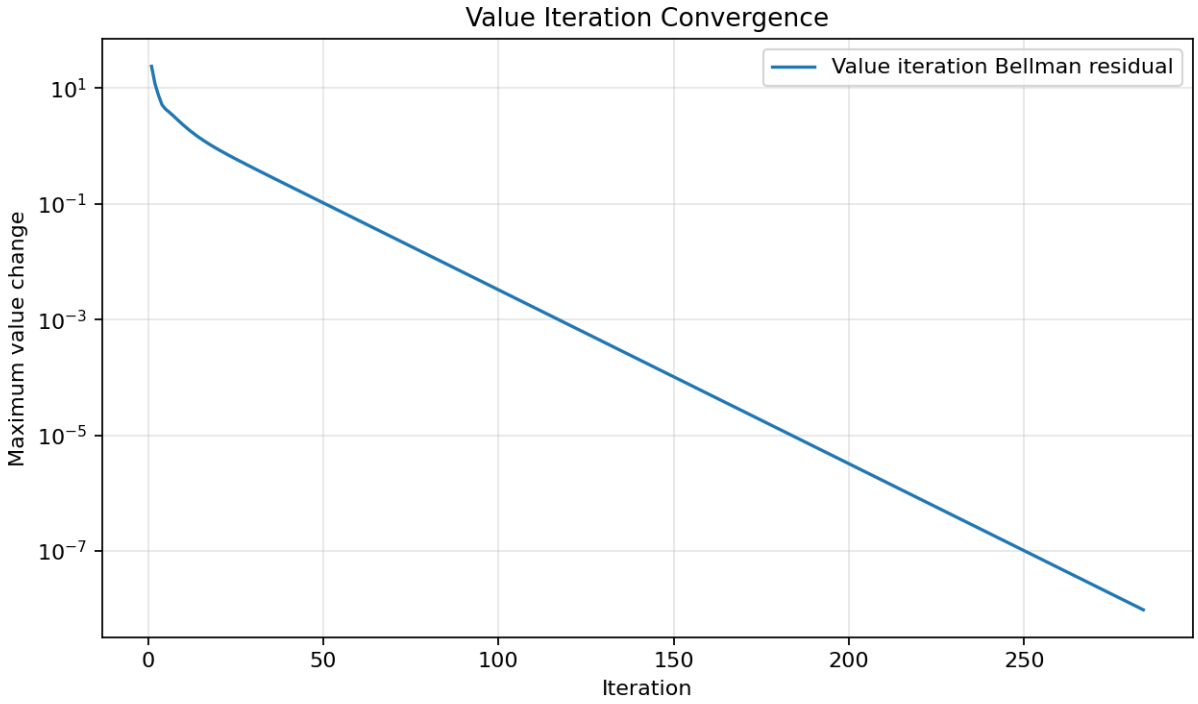

Figure 3.1 - Bellman residual decreases monotonically until the value-iteration stopping tolerance is reached.

2.4 Task 3.1.2 - Policy Iteration

Policy iteration alternates between policy evaluation and policy improvement. For a fixed policy π , policy evaluation solves

$$V^{\pi}(s) = \sum_{s'} P(s' \mid s, \pi(s))[\mathrm{r}(\mathrm{s},\pi(\mathrm{s}),s') + \gamma V^{\pi}(s')]$$

In the implementation, this equation is evaluated iteratively until the policy-evaluation residual is below 1e-10. Then the policy-improvement step is

$$\pi_{\mathrm{new}}(s) \in \arg\max_{a \in A(s)} \sum_{s'} P(s' \mid s, a)[\mathrm{r}(\mathrm{s},\mathrm{a},s') + \gamma V^{\pi}(s')]$$

Starting from the hold policy, the algorithm reached a stable policy after 6 improvement sweeps. The final policy is exactly the same as the value-iteration policy over all 35,343 states, and the maximum absolute value-function difference is only 1.32e-7, which is numerical tolerance error.

Sweep	Policy-evaluation iterations	Changed states after improvement	V(s0) after evaluation
1	350	25830	0.0000
2	350	9368	10.1923
3	350	1737	14.2512
4	350	308	14.2804
5	350	7	14.2808
6	350	0	14.2808

Table 3.2 - Policy iteration convergence from the initial hold policy.

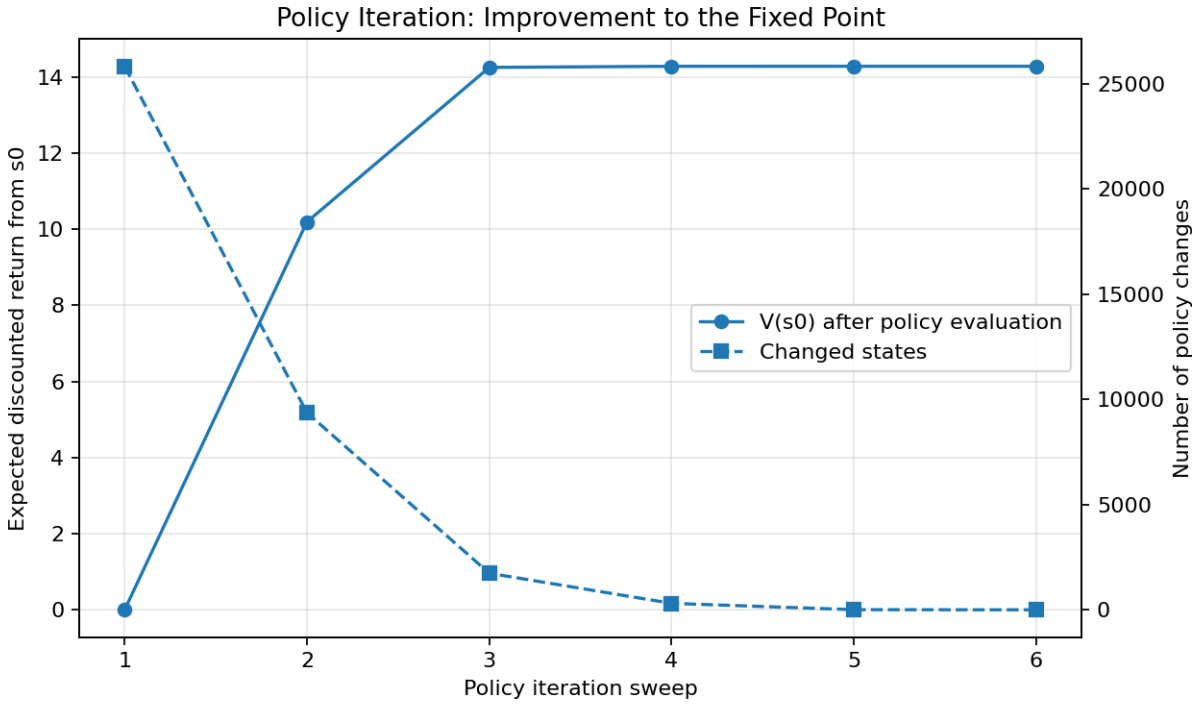

Figure 3.2 - Policy iteration improves the initial-state value and reaches zero policy changes at sweep 6.

2.5 Interpretation of the Optimal Trading Strategy

The optimal policy is not simply 'always buy in Bull' or 'always sell in Bear'. It depends on price, cash, current holdings, and market state. Two qualitative rules appear clearly:

- When the asset is very cheap, the policy buys aggressively in Sideways and Bull markets because the upside is large relative to the price paid.
- When the price is high, the policy usually holds or sells because the price cap limits upside while the absorbing state x=0 creates large downside risk.
- With existing holdings, the Bear-market policy sells most of the position. With no holdings, the Bear-market policy often waits unless the price is extremely low.
- At x=0 all future rewards are zero; tie-breaking may choose selling zero-value assets, but this does not change value or wealth.
- The optimal policy obtained from value iteration and policy iteration demonstrates a cautious investment behavior that reflects the transient nature of the asset price process. Since the price process possesses an absorbing state at $x = 0$, the agent faces an inherent long-term risk of losing all investment value. Therefore, the optimal strategy balances short-term profit opportunities against long-term risk exposure.
- In particular, the policy often favors holding or moderate trading actions rather than aggressive buying, especially when the market state is uncertain or the available budget is limited. This behavior explains why the resulting mean final wealth remains moderate (around the mid-30 range) rather than extremely large values. The objective of the dynamic programming formulation is to maximize the expected discounted cumulative reward rather than the final wealth alone, which naturally encourages stable and risk-aware strategies instead of highly aggressive investment patterns.

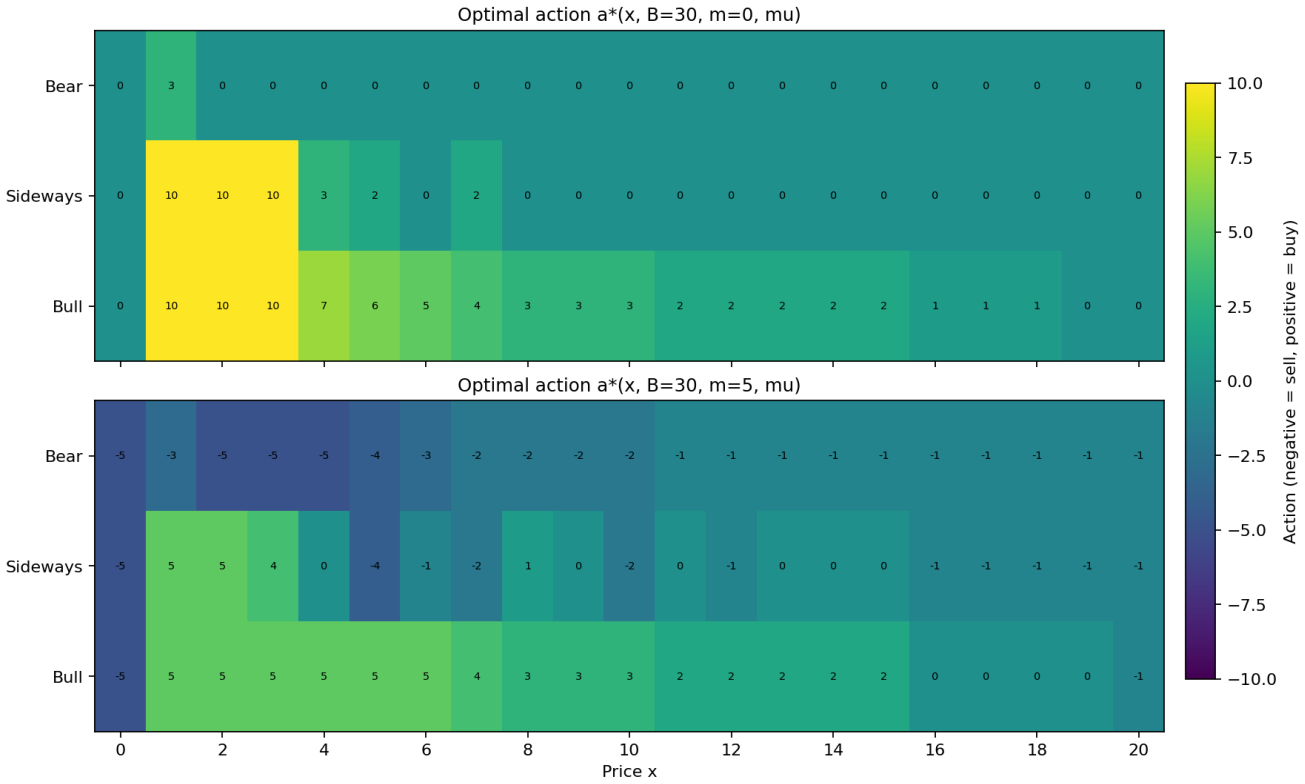

Figure 3.3 - Optimal actions for two representative budget/holding slices. Positive numbers mean buy, negative numbers mean sell.

Market	x=1	x=2	x=3	x=5	x=10	x=15	x=20
Bear, B=30,m=0	3	0	0	0	0	0	0
Sideways, B=30,m=0	10	10	10	2	0	0	0
Bull, B=30,m=0	10	10	10	6	3	2	0

Table 3.3 - Representative optimal actions for the cash-only state slice B=30, m=0.

2.6 Task 3.2 - Experimental Test of the Known-Model DP Policy

The value-iteration and policy-iteration policies are identical, so only one dynamic-programming policy is simulated. The simulation protocol uses the same simulator constructed in Task 2. Each trial starts from s0 = (10,30,0,Sideways), runs for T=2000 steps, and records the discounted return, final wealth, absorption time, and bankruptcy event.

Metric	Expected / Analytical	Empirical Simulation	Interpretation
Discounted return	14.2808	14.2549 ± 0.1408 (95% CI)	Empirical mean matches the Bellman value.
Trials used	-	100,000	Far above the required 50 independent trials.
Bankruptcy rate	estimated by simulation	7.666%	Occurs when aggressive positions lose all wealth.
Mean final wealth	not Bellman objective	34.108880	Final wealth is reported for interpretation, but the optimized criterion is discounted reward.

Table 3.4 - Experimental validation of the known-model dynamic-programming policy.

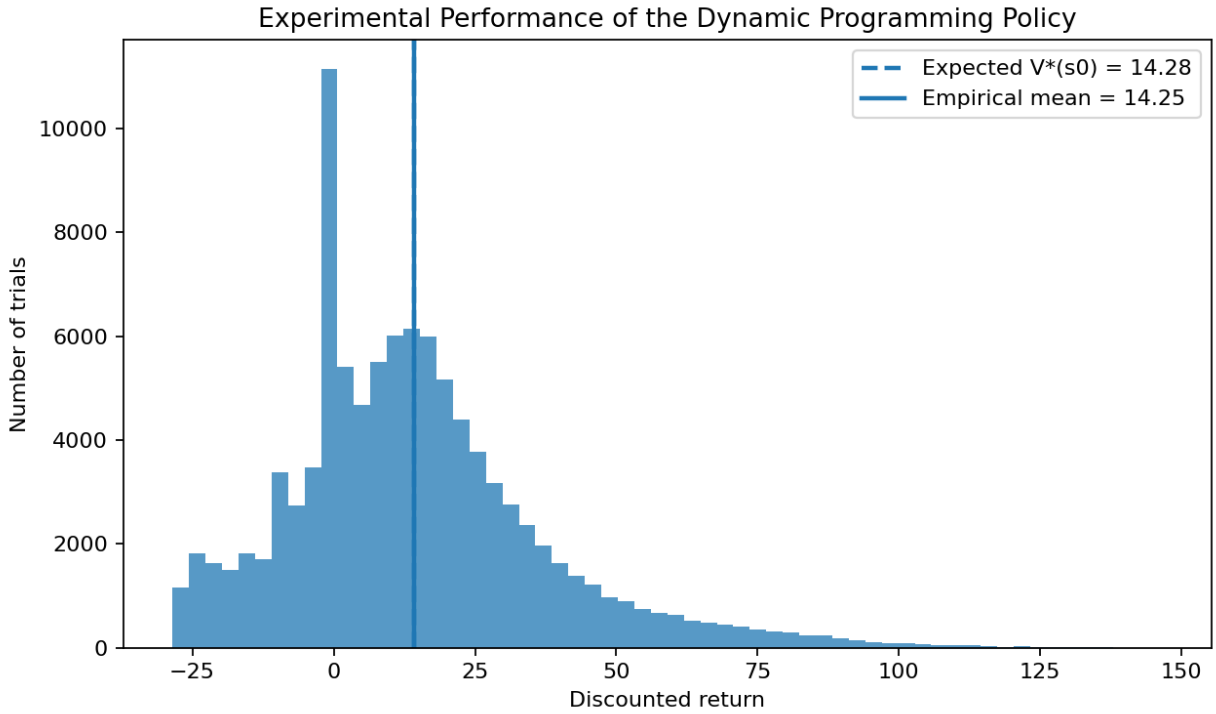

Figure 3.4 - Distribution of discounted returns for 100,000 DP-policy trials compared with the Bellman expected value.

2.7 Task 3.3 - Learning the Model and Repeating Dynamic Programming

For Task 3.3, the transition model is learned from simulator-generated samples rather than from the known probability matrices. The deterministic budget and holdings updates are still known because they are action definitions, but the exogenous kernel

$$K(x', \mu' \mid x, \mu) = P(x_{k+1} = x', \mu_{k+1} = \mu' \mid x_k = x, \mu_k = \mu)$$

is estimated by maximum likelihood from transition counts. Randomized restarts over the 63 exogenous states (x, μ) are used during data generation to avoid the absorbing state x=0 dominating the data. If a row is unobserved in the smallest data set, it is assigned a conservative self-loop with zero useful reward. After estimating K, the complete MDP is reconstructed and value iteration is run again.

Samples	Mean L1 kernel error	Policy agreement	Learned-model V(s0)	True E[return]	Simulator return	Bankruptcy
2,000	0.583	76.06%	20.063	12.290	12.092 ± 0.328	10.44%
10,000	0.221	86.98%	14.9058	13.926	13.777 ± 0.325	9.07%
50,000	0.099	94.15%	13.2627	14.249	14.096 ± 0.317	7.73%
200,000	0.048	96.96%	14.3990	14.258	14.246 ± 0.310	7.355%

Table 3.5 - Effect of model-learning sample size. Simulator return uses 20,000 trials in this sweep.

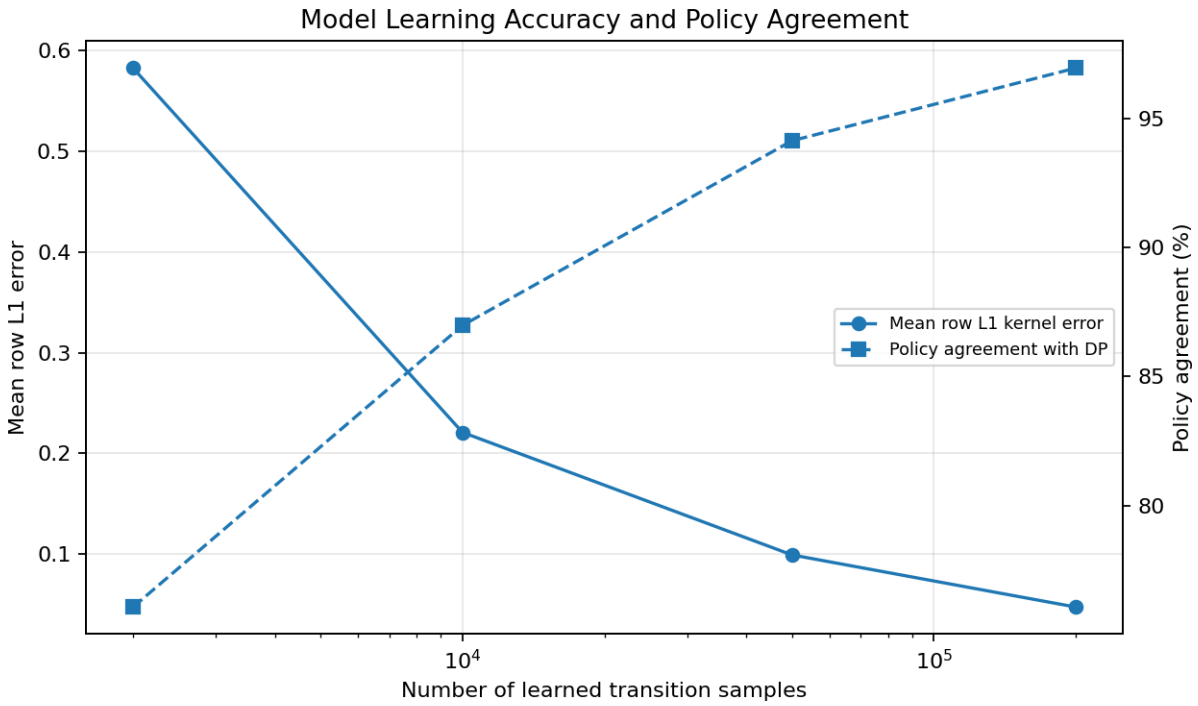

Figure 3.5 - Kernel error decreases and policy agreement with the known-model DP policy increases with more samples.

The model-learning experiment shows the expected bias-variance behavior. With only 2,000 samples, several rare transitions are missing and the learned model is overly optimistic, predicting 22.61 while achieving about 12.09 in the true simulator. With 200,000 samples, the policy agrees with the known-model DP policy on 96.96% of states and its true expected value is 14.2579, almost identical to the optimum 14.2808.

3 Task 4 - Reinforcement Learning Solutions

3.1 Model-Free Learning Setup

Task 4 removes direct use of the transition probabilities during learning. The algorithms observe only simulator transitions $(s_k,\ a_k,\ r_k,\ s_{k+1})$ The same state encoding, feasible-action list, reward definition, and bankruptcy definition are used as in Task 3. The final learned policies are then evaluated in the true simulator with 100,000 independent trials.

Two model-free algorithms are implemented:

- Tabular Q-learning, an off-policy temporal-difference control algorithm.
- A tabular policy-gradient method implemented as actor-critic with a softmax policy and a learned value baseline.

3.2 Task 4.1.1 - Tabular Q-Learning

The Q-learning update is

$$Q(s_k, a_k) \leftarrow Q(s_k, a_k) + \alpha_k \left[r_k + \gamma \max_{a'} \mathrm{Q}(s_{k+1}, a') - \mathrm{Q}(s_k, a_k)\right]$$

Actions are selected by epsilon-greedy exploration during training. The final policy chooses the feasible action with the largest learned Q-value; for unvisited states it defaults to the hold action to avoid arbitrary selling caused by table initialization.

Hyperparameter / Design Choice	Value Used
Training episodes	2,000,000
Training horizon	250 steps per episode; episode ends early after x=0

Exploration	epsilon linearly decays from 0.80 to 0.01
Learning rate	alpha linearly decays from 0.08 to 0.005
Random starts	20% of episodes use random valid states for broader state-action coverage
Final policy	Greedy with respect to Q; unvisited states default to hold

Table 4.1 - Q-learning implementation settings.

3.3 Task 4.1.2 - Tabular Policy Gradient

A direct Monte-Carlo policy-gradient method has high variance in this problem because bankruptcy and absorption create long stretches of zero future reward after a risky early decision. Therefore the final policy-gradient implementation uses tabular actor-critic, which is still a policy-gradient method but replaces the Monte-Carlo return with a one-step TD advantage estimate.

The policy is a softmax over state-action preferences theta(s,a):

$$\pi_\theta(a \mid s) = \frac{\exp\left(\frac{\theta(s,a)}{\tau}\right)}{\sum_{b \in A(s)} \exp\left(\frac{\theta(s,b)}{\tau}\right)}$$

The critic maintains V_w(s). At each step, the TD error is

$$\delta_k = r_k + \gamma V_w(s_{k+1}) - V_w(s_k)$$

The actor and critic updates are

$$\theta(s_k, a_k) \leftarrow \theta(s_k, a_k) + \eta_\theta \, \delta_k \, \nabla \log \pi_\theta(a_k \mid s_k)$$
$$V_w(s_k) \leftarrow V_w(s_k) + \eta_V \, \delta_k$$

The softmax temperature is gradually reduced so that the final policy is nearly deterministic. The actor update uses TD-error clipping at ±100 for numerical stability, which prevents rare very large wealth changes from dominating the preference table.

Hyperparameter / Design Choice	Value Used
Training episodes	2,000,000
Training horizon	250 steps per episode; episode ends early after x=0
Policy	Tabular softmax over feasible actions
Temperature	tau linearly decays from 1.0 to 0.1
Actor learning rate	0.0015 to 0.00005
Critic learning rate	0.03 to 0.001
Stabilization	TD-error clipping at ±100
Final policy	Greedy action with largest learned preference theta(s,a)

Table 4.2 - Tabular actor-critic policy-gradient settings.

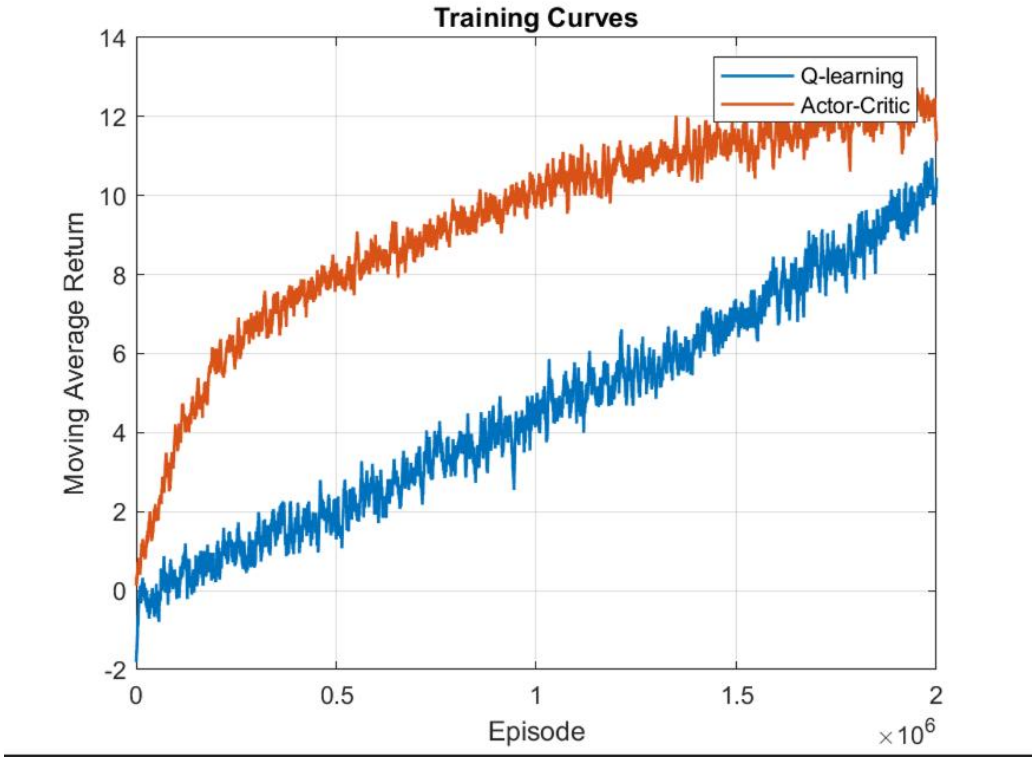

Figure 4.1 - Smoothed training returns. The curves show learning progress during exploratory training; final evaluation is reported separately.

3.4 Task 4.2 - Experimental Comparison of Model-Free Policies

The final Q-learning and actor-critic policies are compared against the dynamic-programming policy. Expected values for the learned policies are computed by evaluating the learned policy in the known MDP after training; empirical values are computed by direct simulation.

Policy	Expected E[discounted return]	Empirical return (100,000 trials)	95% CI half-width	Bankruptcy	Mean final wealth
Dynamic programming optimum	14.281	14.313	0.1408	7.666%	34.109
Learned-model DP (200,000 samples)	14.258	14.502	0.1395	7.355%	34.185
Q-learning	12.758	12.822	0.1228	2.357%	32.776
Policy gradient / actor-critic	12.359	12.385	0.1309	9.992%	29.85

Table 4.3 - Final performance and bankruptcy comparison across solution methods.

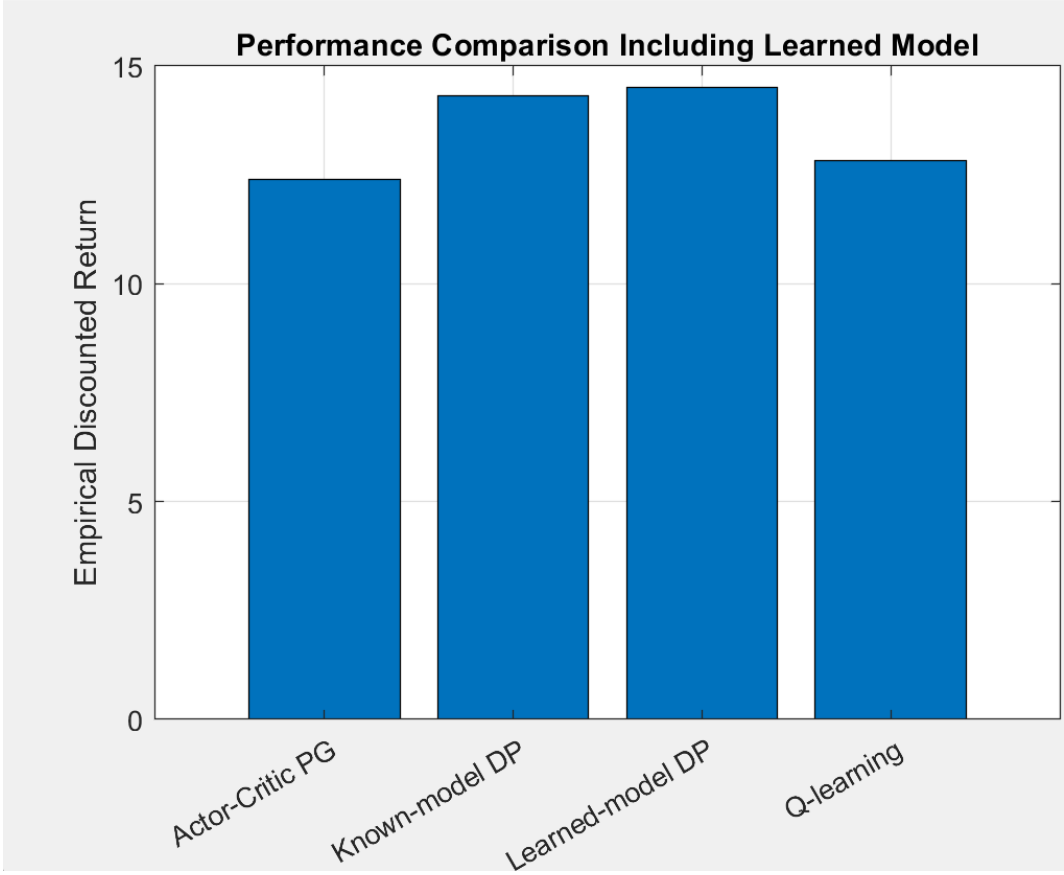

Figure 4.2 - Expected and empirical discounted returns for the final policies.

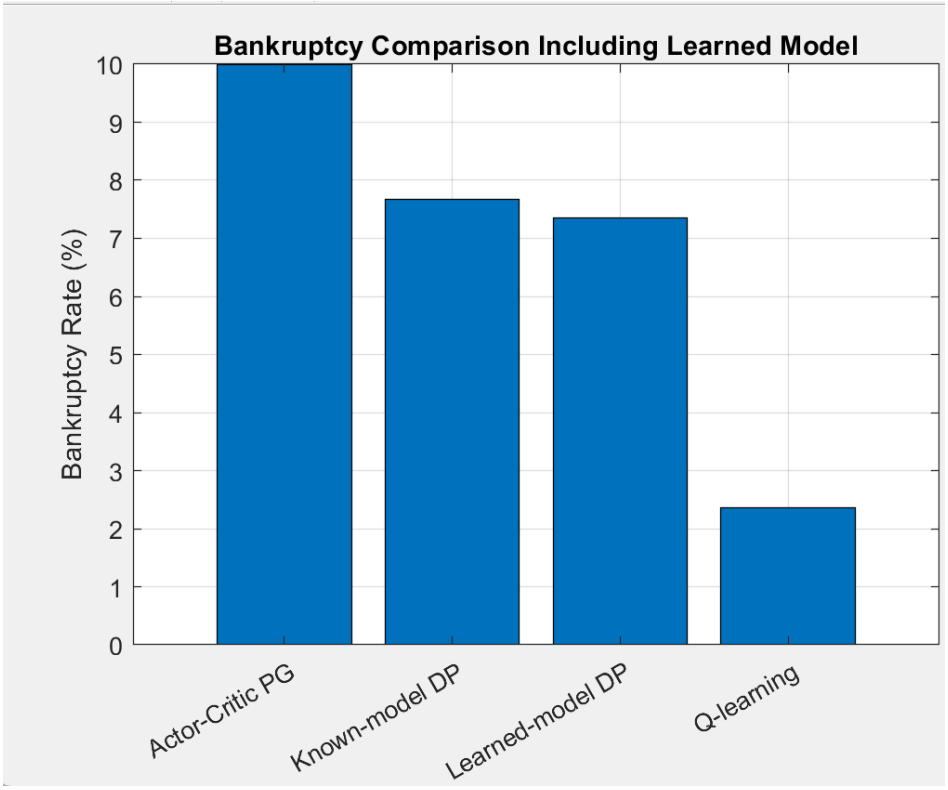

Figure 4.3 - Bankruptcy percentages for the final policies.

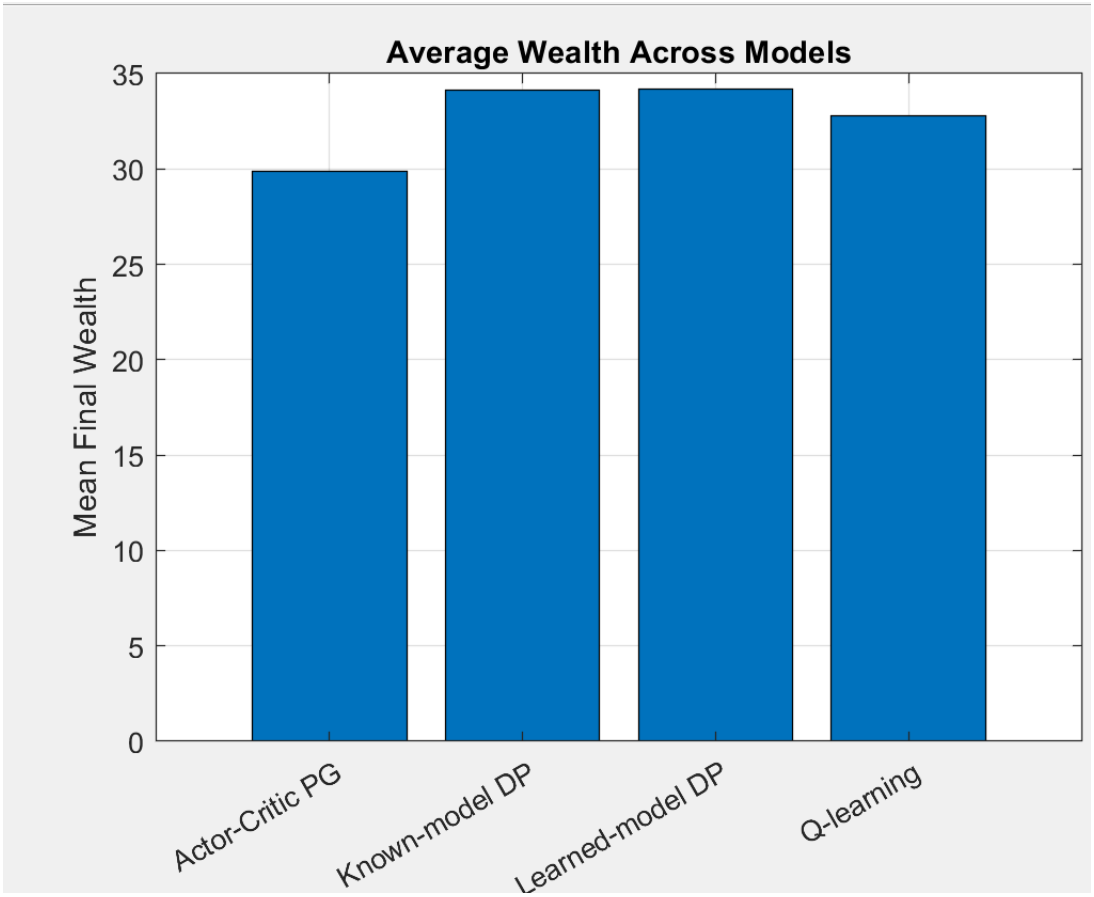

Figure 4.4 – Average Wealth for the final policies.

3.5 Interpretation of the RL Results

The dynamic-programming policy is the benchmark because it uses the exact known model. The learned-model DP policy with 200,000 transition samples almost matches this benchmark, showing that most of the performance loss in model-based learning comes from transition estimation error rather than planning error.

Q-learning learns a safer but more conservative policy. Its expected discounted return is 12.822, which is below the DP value 14.2808, but its bankruptcy rate is only 2.357%. The lower bankruptcy rate is not automatically better because the optimization objective is discounted reward, not bankruptcy minimization. The result means the learned Q-policy sacrifices some profitable risky opportunities.

The policy-gradient actor-critic policy reaches 12.385 expected discounted return. Its bankruptcy rate is 9.996%, way larger than Q-learning and DP. This is consistent with a stochastic-gradient policy-search method that discovers more aggressive profitable actions than Q-learning so its bankruptcy rate is highest among the methods.

The DP policy has the highest return but also higher bankruptcy rate than Q-learning. This is not a contradiction: the reward function values expected discounted wealth change and does not include an explicit risk penalty. A risk-sensitive objective could reduce bankruptcy further, but that would be a different MDP objective.

4 Conclusion

Tasks 3 and 4 were completed using the MDP and simulator established in Report I. The main conclusions are:

- Value iteration converged in 284 sweeps and produced V*(s0) = 14.2808.
- Policy iteration converged in 6 improvement sweeps and produced exactly the same policy as value iteration over all 35,343 states.
- The known-model DP policy achieved an empirical discounted return of 14.2549 ± 0.1408 over 100,000 trials, consistent with the analytical Bellman value.
- A model learned from 200,000 simulator transition samples produced a policy with true expected return 14.2579, almost matching the exact DP policy.
- Model-free Q-learning achieved lower return than DP, 12.822, but also the lowest bankruptcy rate, 2.287%.
- The tabular actor-critic policy-gradient method achieved 12.385 expected return and 9.992% bankruptcy, regarding it is the worse method among the other methods
- Overall, multiple solution methods were implemented and compared, including value iteration, policy iteration, model-based dynamic programming with a learned model, Q-learning, and Actor–Critic policy gradient methods. Each algorithm produced distinct policies and performance characteristics due to differences in how they estimate value functions and update decision rules.
- The dynamic programming solutions based on the known model consistently achieved the highest expected discounted return and stable empirical performance, indicating that these methods produced the optimal strategy under the given Markov decision process formulation. The learned-model dynamic programming approach achieved nearly identical performance, demonstrating that the transition model estimated from simulated trajectories was sufficiently accurate.
- In contrast, model-free reinforcement learning methods such as Q-learning and Actor–Critic exhibited slightly lower performance and higher variability, particularly in the case of the Actor–Critic method, which showed increased bankruptcy rates due to the stochastic nature of policy-gradient updates. Overall, these results confirm that the optimal policy obtained through dynamic programming provides the most reliable performance, while reinforcement learning methods offer flexible alternatives when the system model is unknown.
- .

5 Appendix A - Implementation Checks

Check	Result
Feasible actions always keep B in [0,50] and m in [0,10]	Passed
Price always remains in [0,20]	Passed
Noise distribution depends on current market state, not next market state	Used consistently
Reward uses post-trade holdings m+a	Used consistently
x=0 is absorbing and produces zero future reward	Used consistently
Value iteration and policy iteration policies agree	100% agreement
Simulation trial count for final comparison	100,000 independent trials

Table A.1 - Consistency checks used to avoid implementation errors.

The accompanying MATLAB code file follows the same indexing convention and algorithmic structure described in this report.